

Mejora de los modelos de calificación crediticia mediante máquinas de vectores de soporte

Segunda entrega — Fase 2: datos, modelamiento y validación

Integrantes:

Daniela González Ortiz

Belén Shalom Dueñas Pacheco

Jhabes Esteban Rodríguez Enríquez

Curso:

Machine Learning II

Universidad Externado de Colombia

2026

1. Consolidación de los datos

El conjunto definitivo es una **simulación fundamentada** de 6.500 solicitantes de crédito de consumo en Colombia, generada con semilla fija (42) por `src/generar_datos_simulados.py`. Reproduce la estructura de variables del German Credit Dataset que usan Xu y Cheng, adaptada a nombres, unidades (COP) y categorías locales: edad, género, tipo de ocupación (formal o informal), tipo de vivienda, nivel de ahorro, saldo en cuenta, monto, plazo, propósito y estado de pago de créditos anteriores. Simulamos porque la información crediticia individual está protegida por la Ley 1266 de 2008 y no existe en repositorios públicos.

Cómo se generó. Cada variable sigue una distribución calibrada contra su análoga del German Credit: la edad y el plazo, distribuciones Gamma desplazadas (asimétricas a la derecha); el monto, una log-normal que depende del plazo y del propósito, para reproducir la correlación plazo-monto del original; y las categóricas, proporciones tomadas de checking, savings y credit_history. La informalidad (44%) y la tenencia de vivienda son supuestos del contexto colombiano y se documentan como tales. El riesgo se construyó como una función logística de un puntaje latente que combina todas las variables con la dirección esperada (no tener cuenta, la mora previa, el monto y el plazo aumentan el riesgo), más un ruido que representa factores no observados; el intercepto se calibró para obtener 22.5% de clientes "malos", dentro del rango de 20%-25% fijado en la Primera entrega. Una decisión de adaptación merece explicitarse: en el German Credit, "sin cuenta corriente" se asocia a *menor* riesgo incluso controlando por las demás variables (Sección 3), así que la dirección contraria que usamos es un supuesto para Colombia, donde no tener cuenta se asocia a informalidad y a falta de historial en centrales de riesgo, y no algo que el dataset alemán respalde.

Verificación. `src/validar_simulacion.py` comprueba 14 criterios de aceptación, todos cumplidos (Tabla 1). La Figura 1 muestra que las distribuciones simuladas de edad y plazo se superponen con las originales y que el monto, una vez estandarizado, tiene la misma forma asimétrica. La Figura 2 confirma que la tasa de "malo" crece de forma estrictamente monótona en el orden de riesgo esperado: del 9.8% con saldo alto al 30.1% sin cuenta, y del 13.0% con historial al día al 37.9% con mora severa.

Tabla 1. Verificación de la simulación frente a la estructura prevista y al German Credit.

Criterio	Simulado	German Credit / criterio	¿Cumple?
Registros / proporción de "malo"	6.500 / 22.5%	≥ 5.000 / 20%-25%	Sí
Edad: media / desviación	35.5 / 11.0 años	35.5 / 11.4 años	Sí
Plazo: media / desviación	20.9 / 11.4 meses	20.9 / 12.1 meses	Sí
Monto: coeficiente de variación	0.83	0.86	Sí
Correlación plazo-monto	0.60	0.62	Sí
Monotonicidad de 6 relaciones con el riesgo	6 de 6	Crecimiento estricto	Sí
Faltantes por columna (base de trabajo)	2.9%-5.0%	3%-5%	Sí

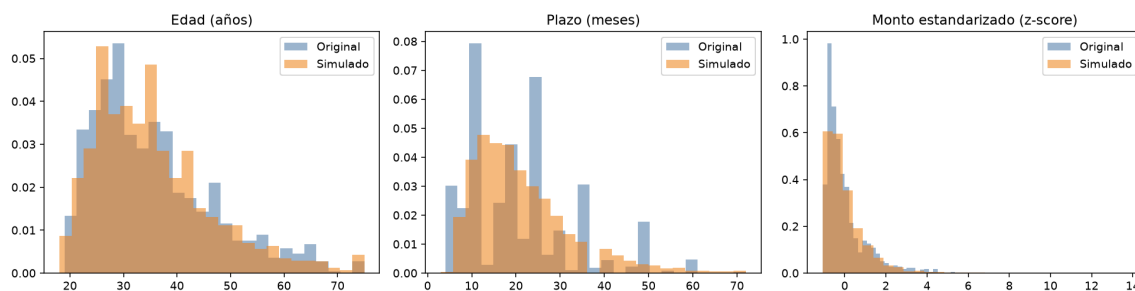


Figura 1. Distribuciones de edad, plazo y monto estandarizado: German Credit (azul) frente a la simulación (naranja). La forma y la dispersión relativa se reproducen; la cola del monto simulado es algo más larga (asimetría 2.8 frente a 1.95), propia de la log-normal.

Calidad y transformaciones. El modelamiento usa la versión con defectos controlados, que replica una base real de solicitudes: entre 2.9% y 5.0% de faltantes por columna (22% de las filas tiene al menos uno) y ruido de medición del 2% en cerca del 3% de los registros. Los faltantes del saldo son MAR: solo ocurren en clientes con cuenta y son más frecuentes en saldos bajos (12.1% frente a 3.5%), lo que justifica imputarlos con la mediana de su categoría de saldo y no con la mediana global. La base cumple las 16 reglas de consistencia revisadas (sin duplicados, categorías inválidas, valores fuera de rango ni contradicciones entre el saldo y su categoría). Los atípicos del criterio IQR (8.8% en saldo y 5.1% en monto) se conservan porque son consecuencia legítima de la forma log-normal. Las transformaciones necesarias son, entonces, imputación, escalamiento, codificación y tratamiento del desbalance.

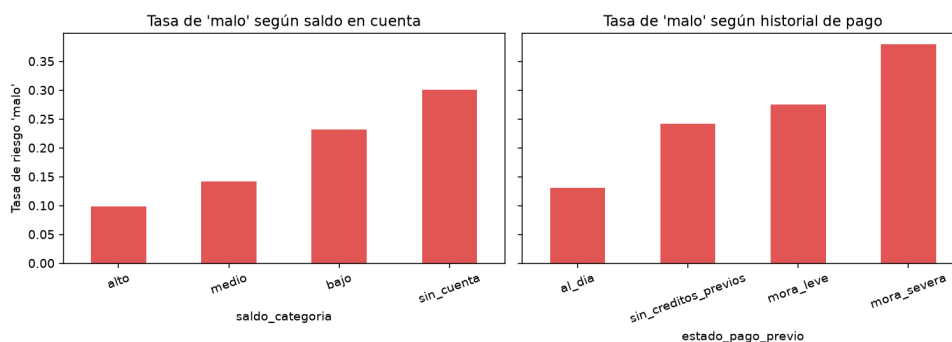


Figura 2. Tasa de clientes "malos" por saldo en cuenta y por historial de pago en la base simulada: ambas relaciones son monótonas en la dirección documentada en la Primera entrega.

2. Flujo reproducible sin fuga de datos

Todo el preprocesamiento vive en un único Pipeline, definido en el módulo `src/preprocesamiento.py` e importado por todos los notebooks, de modo que cada modelo recibe exactamente las mismas transformaciones. Partimos los datos en 80% de entrenamiento y 20% de prueba, estratificando por la clase y con semilla fija. El orden de las etapas es: (1) imputación del saldo por categoría y del resto por mediana o moda; (2) escalamiento de las numéricas; (3) balanceo opcional con SMOTENC; (4) codificación one-hot; (5) selección de variables opcional con LASSO; y (6) el modelo.

Este orden evita la fuga por tres razones. Primero, **cada etapa se ajusta solo con los datos de entrenamiento de cada partición**: medianas, medias de escalamiento, categorías del one-hot y coeficientes del LASSO se aprenden en fit y en validación o prueba solo se aplican. Si se imputara o escalara con toda la base antes de partir, el modelo vería información de los datos con que luego se evalúa. Segundo, **el balanceo y la selección están dentro del Pipeline**: los ejemplos sintéticos se generan solo en fit y nunca en predict, y las variables se seleccionan sin ver los datos de validación, que es una de las formas más comunes de fuga. Tercero, el orden responde a requisitos técnicos: SMOTENC no admite faltantes (por eso se imputa antes), busca vecinos por distancia (por eso se escala antes) y debe ver las categóricas como categorías y no como columnas 0/1 (por eso se codifica después). Para el desbalance comparamos tres estrategias con validación cruzada sobre entrenamiento: ponderar las clases (`class_weight="balanced"`) logró el mismo ROC-AUC que no balancear (0.765) y más recall de "malo" que SMOTENC (0.695 frente a 0.632), así que es la estrategia que usan todos los modelos.

3. Reproducción y adaptación del método

Reproducción en el escenario del artículo. Antes de adaptar el método lo reproducimos sobre el German Credit, con la configuración del artículo: SVM y regresión logística sin pesos de clase, con hiperparámetros elegidos por accuracy en validación cruzada de 5 folds. El SVM con kernel gaussiano (RBF) alcanza una accuracy de 75.5%, igual al 75% que reportan los autores (Tabla 2). La ventaja sobre la regresión logística, en cambio, no se reproduce: con las variables escaladas y la regularización ajustada, la logística llega a 74.8% y no a 70%, y la diferencia no es significativa. El 70% reportado coincide con la accuracy de predecir siempre "bueno", lo que sugiere que la logística del artículo no se preparó en igualdad de condiciones. Los coeficientes de nuestro SVM lineal coinciden con las variables que el artículo destaca: el saldo de la cuenta es la más importante y la duración la segunda, y cuatro de las cinco variables del artículo están entre nuestras siete principales.

Tabla 2. Reproducción sobre el German Credit (accuracy en validación cruzada de 5 folds, media $\pm$ desviación). El p-valor compara el SVM RBF con la logística mediante la prueba t corregida de Nadeau y Bengio.

Modelo	Accuracy reportada (artículo)	Nuestra reproducción	Hiperparámetros
SVM RBF	75%	75.5% $\pm$ 2.8%	$C = 3, \gamma = 0.1$
SVM lineal	—	74.8% $\pm$ 3.0%	$C = 0.1$
Regresión logística	70%	74.8% $\pm$ 4.3%	$C = 0.1$

Diferencia SVM RBF – logística: +0.8 puntos ($p = 0.76$). Predecir siempre "bueno" da 70.0%.

Adaptación a la base colombiana. Conservamos lo esencial del método: el SVM con kernel RBF como modelo principal, el SVM lineal como versión interpretable, la regresión logística como referencia y el ajuste de C y γ por validación cruzada. Cambiamos lo que exigen los datos y el problema local: la base simulada en COP con variables del mercado colombiano; el tratamiento de faltantes dentro del Pipeline; los pesos de clase, porque el desbalance es de 22.5%; la selección de hiperparámetros por ROC-AUC y no por accuracy, porque la accuracy premia predecir "bueno"; y un umbral de decisión que minimiza el costo esperado con la matriz 5:1 del German Credit (aprobar a un cliente malo cuesta cinco veces más que rechazar a uno bueno), elegido también por validación cruzada en entrenamiento. Estos cambios modifican los datos y el procedimiento de validación, pero

no el propósito del método: un clasificador de riesgo de margen máximo, capaz de capturar no linealidades con el kernel.

En la base simulada, el SVM RBF alcanza su mejor ROC-AUC con $c = 1$ y $\gamma = 0.01$ (0.767 ± 0.015), dentro de una meseta amplia de valores casi equivalentes. Las curvas de validación (Figura 4) muestran que γ es el hiperparámetro más sensible: por encima de 0.03 el AUC de entrenamiento se dispara hacia 1 mientras el de validación cae hasta 0.665, una señal clara de sobreajuste. Para comprobar si el kernel del artículo es el adecuado comparamos los cuatro kernels estándar con sus propias mallas y los mismos folds (Figura 3): el gaussiano es nominalmente el mejor en las dos bases, pero su ventaja sobre el lineal, el polinómico y el sigmoide no supera 0.007 de AUC ni es significativa ($p \geq 0.10$). Como la base simulada genera el riesgo con un puntaje aditivo, no hay estructura no lineal que un kernel más flexible pueda aprovechar.

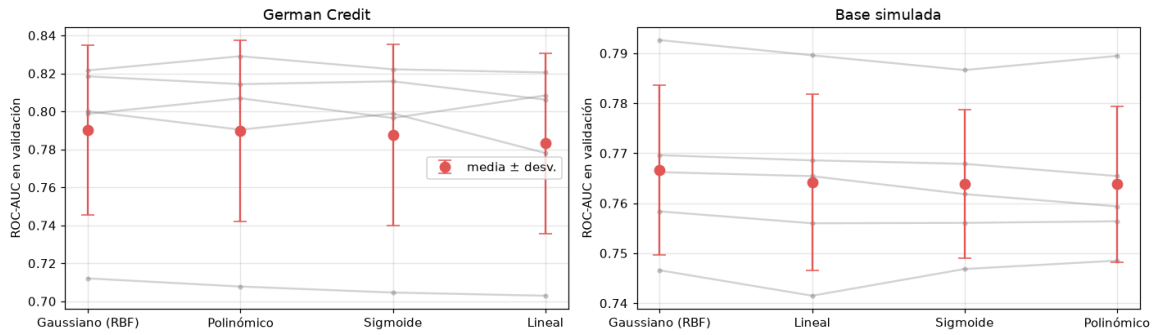


Figura 3. ROC-AUC de validación por kernel en el German Credit y en la base simulada. Las líneas grises son los mismos 5 folds para todos los kernels: la variación entre folds supera con creces la diferencia entre kernels.

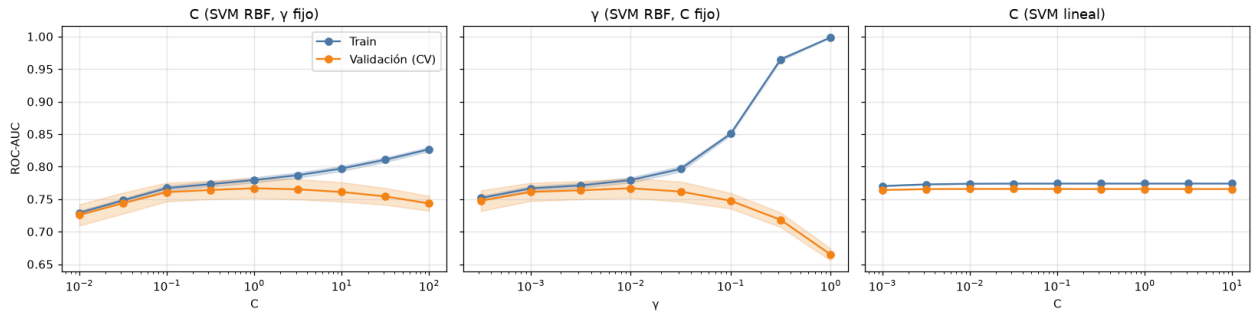


Figura 4. Curvas de validación del SVM (media ± 1 desviación entre folds). En el RBF, la brecha entre entrenamiento y validación crece con C y sobre todo con γ ; el SVM lineal es estable en todo el rango de C .

Comparación con otros modelos. Siguiendo la comparación del artículo, entrenamos además un árbol de decisión, Random Forest y Gradient Boosting (`HistGradientBoostingClassifier`), cada uno con su espacio de hiperparámetros, y evaluamos el SVM RBF con selección de variables por LASSO, una de las técnicas de selección del artículo. No incluimos modelos de densidad ni clustering porque el método original es supervisado y no los emplea. Todos los modelos se comparan entre sí y frente a dos líneas base, el clasificador de mayoría y la regresión logística, en la Sección 4.

4. Validación, métricas e interpretación preliminar

Estrategia de validación. Para evitar que la selección de hiperparámetros infle el desempeño, usamos **validación cruzada anidada** sobre los 5.200 clientes de entrenamiento: un ciclo externo de 5 folds estima el desempeño y, dentro de cada fold externo, un ciclo interno de 3 folds elige los hiperparámetros y el umbral de costo mínimo. Cada cifra de la Tabla 3 proviene de clientes que no participaron en ninguna decisión del modelo. El conjunto de prueba se usó una sola vez, al final, para confirmar. Para comparar modelos usamos la prueba t pareada corregida de Nadeau y Bengio, que tiene en cuenta que los folds comparten datos de entrenamiento.

Métricas. Como definimos en la Primera entrega, las métricas principales son el **F1** y el **ROC-AUC** de la clase "malo", complementadas con su **recall**. El ROC-AUC mide la capacidad de ordenar a los clientes por riesgo sin depender del umbral, por eso guía la elección de hiperparámetros. El recall refleja directamente el error más caro (aprobar a un cliente malo) y el F1 evita premiar a un modelo que rechaza a casi todos. Reportamos también el costo medio con la matriz 5:1, y la accuracy solo como referencia: con 22.5% de "malos", el clasificador de mayoría tiene 77.5% de accuracy sin detectar a nadie. Como los datos son simulados, añadimos un **techo teórico**: la probabilidad verdadera $P(\text{malo} \mid x)$ de la estructura generadora, el mejor desempeño que puede lograr cualquier modelo con estas variables.

Tabla 3. Validación cruzada anidada (5 folds externos $\times$ 3 internos; media $\pm$ desviación entre folds externos). F1, recall y costo se calculan con el umbral de costo mínimo elegido en el ciclo interno. El p-valor compara el ROC-AUC de cada modelo con la regresión logística.

Modelo	F1 (malo)	ROC-AUC	Recall (malo)	Costo medio 5:1	p-valor (AUC)
Mayoría (línea base)	0.000	0.500	0.000	1.123 ± 0.003	< 0.001
Regresión logística (línea base)	0.482 ± 0.016	0.765 ± 0.018	0.840 ± 0.029	0.550 ± 0.028	—
SVM lineal	0.485 ± 0.010	0.765 ± 0.017	0.823 ± 0.033	0.552 ± 0.025	0.99
SVM RBF	0.490 ± 0.013	0.766 ± 0.017	0.832 ± 0.025	0.541 ± 0.016	0.29
SVM RBF + selección LASSO	0.488 ± 0.013	0.765 ± 0.017	0.820 ± 0.012	0.549 ± 0.022	0.60
Árbol de decisión	0.442 ± 0.021	0.708 ± 0.013	0.776 ± 0.075	0.645 ± 0.030	0.04
Random Forest	0.475 ± 0.011	0.755 ± 0.011	0.800 ± 0.022	0.578 ± 0.016	0.15
Gradient Boosting	0.478 ± 0.018	0.762 ± 0.018	0.833 ± 0.029	0.560 ± 0.036	0.47
<i>Techo teórico: $P(\text{malo} \mid x)$</i>	0.497 ± 0.016	0.775 ± 0.019	0.818 ± 0.025	0.536 ± 0.033	—

Resultados. Todos los modelos superan con amplitud al clasificador de mayoría, cuyo costo (1.12 por cliente) duplica el de cualquier otro: el modelo con mejor accuracy es el peor de todos. Con la regla fijada antes de ver los resultados (mayor ROC-AUC medio y, en empate, menor costo), el **mejor modelo es el SVM RBF**, el método del artículo: tiene el mejor ROC-AUC, el mejor F1 y el menor costo. Sin embargo, su ventaja sobre la regresión logística es de solo +0.001 de AUC ($p = 0.29$), y tampoco es significativa frente al SVM lineal, el SVM con LASSO ni el Gradient Boosting. Solo supera de forma significativa al árbol de decisión, en AUC y en costo, y al Random Forest en costo ($p = 0.001$). La Figura 5 explica por qué: las líneas de cada fold son casi paralelas, así que la variación entre particiones (≈ 0.017 de AUC) es mucho mayor que la diferencia entre modelos. Todos, salvo el árbol, quedan a 0.02 de AUC o menos del techo teórico: la señal disponible en estas variables está prácticamente agotada. Que los árboles y ensambles no ganen es coherente con una base cuyo riesgo

es aditivo y sin interacciones; el Gradient Boosting, de hecho, elige árboles de un solo corte en 4 de 5 folds. La selección por LASSO conserva 30 de 32 columnas y no mejora al SVM, porque con 11 variables casi todas relevantes hay poco que descartar.

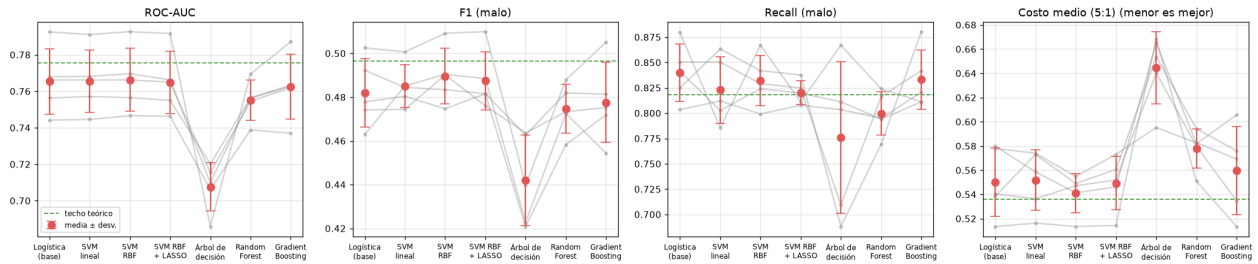


Figura 5. Validación anidada: ROC-AUC, F1, recall y costo por fold externo (líneas grises, los mismos folds para todos los modelos), media $\pm$ desviación (rojo) y techo teórico (verde).

Estabilidad. El procedimiento es estable: el SVM RBF elige $c = 1$ y $\gamma = 0.01$ en 4 de 5 folds externos (el quinto, un vecino de la misma meseta), el umbral de costo mínimo varía poco entre folds y las métricas en prueba confirman las de la validación anidada. En prueba, el SVM RBF alcanza un ROC-AUC de 0.776 (intervalo bootstrap del 95%: 0.747-0.804), igual al techo teórico, con recall de 0.866 y costo de 0.536 (Tabla 4). El Gradient Boosting cuesta algo menos en esta partición (0.525), pero la diferencia está dentro del ruido de una sola muestra de prueba, cuyo intervalo de costo tiene un ancho de unas 0.10 unidades; por eso la selección se basa en la validación anidada. El único procedimiento inestable es el árbol de decisión, cuya profundidad elegida varía de 4 a 10 entre folds.

Tabla 4. Evaluación única en el conjunto de prueba (1.300 clientes, 292 "malos"), con los hiperparámetros y el umbral elegidos en todo el entrenamiento.

Modelo	F1 (malo)	ROC-AUC	Recall (malo)	Costo medio 5:1	Accuracy (ref.)
Regresión logística (línea base)	0.488	0.774	0.842	0.539	0.602
SVM lineal	0.483	0.774	0.846	0.545	0.594
SVM RBF	0.483	0.776	0.866	0.536	0.584
SVM RBF + selección LASSO	0.501	0.776	0.798	0.539	0.642
Árbol de decisión	0.473	0.737	0.784	0.586	0.608
Random Forest	0.466	0.770	0.853	0.572	0.561
Gradient Boosting	0.493	0.773	0.856	0.525	0.604
<i>Techo teórico: $P(\text{malo} x)$</i>	0.502	0.776	0.836	0.520	0.628

Tipos de error. Con el umbral 5:1, el SVM RBF detecta a 253 de los 292 clientes malos de prueba a cambio de rechazar a la mitad de los buenos (Figura 6): rechazar a un buen cliente cuesta 1 y aprobar a uno malo cuesta 5, así que conviene rechazar ante la duda, y por eso los falsos positivos aportan el 72% del costo total. Los 39 falsos negativos se concentran en clientes con historial al día (33 de 39), el segmento con menor tasa de mora, y piden créditos más cortos y pequeños. El 85% de ellos también los aprobaría la regla óptima con la estructura verdadera, y todos los comete también algún otro modelo: son errores en gran parte irreducibles con estas variables.

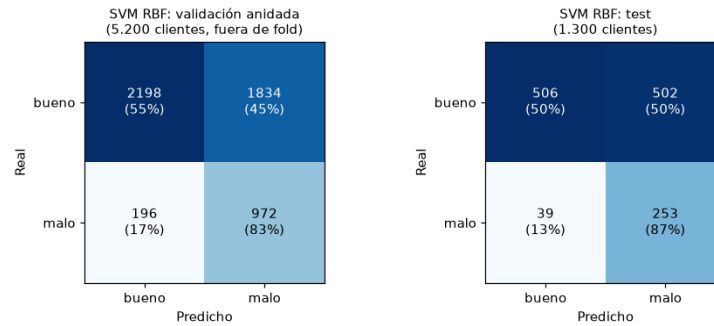


Figura 6. Matrices de confusión del SVM RBF en la validación anidada (5.200 clientes, fuera de fold) y en prueba (1.300 clientes); entre paréntesis, el porcentaje sobre la clase real. Las proporciones coinciden en ambas, lo que indica que el comportamiento no depende de una partición favorable.

Variables relevantes. Como el SVM RBF no tiene coeficientes interpretables, medimos la importancia de cada variable por permutación (la caída del ROC-AUC al desordenarla) en cada fold externo (Figura 7). El historial de pago es, de lejos, la más importante; le siguen el plazo y la categoría de saldo, y después la ocupación, el monto y la vivienda. El ranking es estable entre folds (correlación de Spearman media de 0.89) y coincide con el artículo en cuatro de sus cinco variables destacadas (historial, duración, saldo y monto). El género no influye, lo cual es una buena señal en una variable sensible. Al permutar en grupo las variables redundantes, el orden queda: historial de pago, condiciones del crédito (plazo y monto) y cuenta bancaria. Además, la importancia del SVM RBF se parece mucho a la importancia verdadera de la simulación (Spearman 0.92, solo superado por su versión con LASSO, 0.96), lo que confirma que el modelo aprendió la estructura correcta.

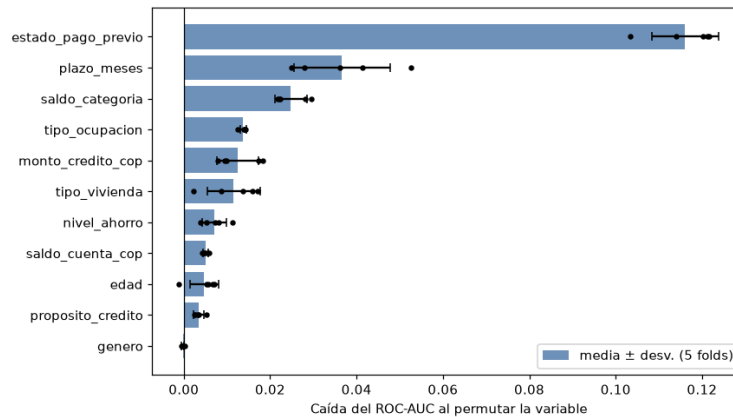


Figura 7. Importancia por permutación del SVM RBF (caída del ROC-AUC), medida en cada fold externo de la validación anidada; los puntos negros son los folds individuales.

Implicaciones. Para el contexto colombiano, el resultado tiene tres lecturas. Primero, la política que induce el modelo rechaza al 94% de los buenos clientes con mora severa y al 61% de los no bancarizados: es coherente con la matriz de costos, pero tiene consecuencias de inclusión financiera que una entidad debería decidir explícitamente. Segundo, los errores del modelo no se reducirían con un algoritmo más complejo sino con información nueva, como ingresos, endeudamiento en centrales de riesgo o comportamiento transaccional. Tercero, como el SVM no supera de forma significativa a la regresión logística, en la Entrega final la elección del modelo debería sopesar su pequeña ventaja frente a la interpretabilidad de los modelos lineales.

5. Conclusiones y limitaciones

Reprodujimos el desempeño del SVM del artículo sobre el German Credit (75.5% frente a 75%) y lo adaptamos a una base simulada colombiana que reproduce fielmente la estructura prevista. Con validación cruzada anidada y un flujo sin fuga, el SVM RBF resulta el mejor modelo (ROC-AUC 0.766 ± 0.017 ; F1 0.490 ± 0.013 ; recall 0.832 ± 0.025), pero sin ventaja significativa sobre la regresión logística, y solo un árbol de decisión queda claramente por debajo. La principal limitación es la circularidad de la simulación: el riesgo se generó como función de los mismos predictores, por lo que los modelos se acercan a un techo conocido y los resultados miden qué tan bien recuperan la estructura diseñada, no su desempeño con datos reales. La estructura aditiva de la base, además, limita la ventaja que pueden mostrar el kernel RBF y los ensambles frente a un modelo lineal.

Código y reproducibilidad

Todo el código está en el repositorio github.com/danielaestudio02-del/svm_credicio_col: los datos se regeneran con `src/generar_datos_simulados.py`, la simulación se verifica con `src/validar_simulacion.py` y los notebooks 01 a 05 reproducen, en orden, las secciones 1 a 4 de este informe, con semilla fija y versiones de librerías fijadas en `requirements.txt`.

Uso de inteligencia artificial generativa

El equipo utilizó herramientas de inteligencia artificial generativa como apoyo en la revisión del código, la depuración y la redacción. Todas las decisiones metodológicas y los resultados fueron revisados por el equipo, que puede sustentarlos.

Referencias

- Hofmann, H. (1994). *Statlog (German Credit Data)* [Conjunto de datos]. UCI Machine Learning Repository.
- Lemaître, G., Nogueira, F. y Aridas, C. K. (2017). Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research*, 18(17), 1-5.
- Ley 1266 de 2008. Por la cual se dictan las disposiciones generales del hábeas data. Congreso de la República de Colombia.
- Nadeau, C. y Bengio, Y. (2003). Inference for the generalization error. *Machine Learning*, 52(3), 239-281.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- Xu, J. y Cheng, Y. (s. f.). *Credit scoring models enhancement using support vector machines*.